

MERN Stack Internship

DAY 6 ASSIGNMENT REPORT

Name: NIKHIL MANDLOI
Course: BTech CSIT 1 year
Position : MERNStackINTERN
Allocated Date: 6/06/26
Submission Date: 6/006/26

MERN Stack Internship – Day 6 Report

Assignment Overview

This report contains the complete submission for Day 6 of the MERN Stack Internship. The evaluation focused on validating the production-ready React application, reviewing frontend-backend data synchronization, verifying database connectivity, understanding React Hooks, and demonstrating code adaptability through endpoint refactoring and live code walkthroughs.

Task 6.1 – Code Defense & Refactoring

Objective

The objective of this task was to demonstrate a clear understanding of the MERN application developed during previous internship sessions. The evaluation focused on explaining React Hooks, API communication, database integration, production build validation, and modifying existing endpoint mappings to demonstrate adaptability.

Concepts Learned

React State Management

React uses the `useState` Hook to store and manage dynamic data inside components.

Example:

```
const [students, setStudents] = useState([]);
```

Purpose:

- Stores application data
 - Updates UI dynamically
 - Triggers component re-rendering
-

`useEffect` Hook

The `useEffect` Hook is used to execute side effects such as API calls after a component renders.

Example:

```
useEffect(() => {  
  
  fetch(`${import.meta.env.VITE_API_URL}/students`)  
    .then((response) => response.json())  
    .then((data) => {  
      setStudents(data);  
    });  
  
}, []);
```

Workflow:

Component Loads

↓

useEffect Executes

↓

API Request Sent

↓

Data Received

↓

State Updated

↓

UI Re-rendered

Frontend-Backend Data Synchronization

The project uses the Fetch API to communicate with the Express backend and retrieve data from MongoDB.

Data Flow:

MongoDB



Mongoose



Express API



CORS Middleware



React Fetch API



useState



Browser UI

Production Build Validation

The React application build generated during Day 5 was reviewed and validated.

Build Command:

```
npm run build
```

Output:

dist/

├── index.html

└── assets/

Benefits:

- Optimized application files
- Faster loading speed
- Reduced file size

- Deployment readiness
-

Database Connectivity Validation

Database connectivity was reviewed using Mongoose.

Example:

```
mongoose.connect(MONGO_URI)
```

Workflow:

Server Starts

↓

Mongoose Connects

↓

MongoDB Atlas

↓

Database Ready

Successful Connection Output:

MongoDB Connected

Implementation

The existing MERN application was reviewed and explained during the evaluation session.

The following areas were validated:

- React useState Hook
- React useEffect Hook
- Fetch API communication
- Express route handling
- MongoDB integration
- Production build generation

- Environment variable usage
 - Error handling middleware
 - Frontend-backend synchronization
-

Endpoint Refactoring Demonstration

Original Endpoint:

```
app.get("/students", async (req, res) => {
```

```
    const students = await Student.find();
```

```
    res.json(students);
```

```
});
```

Refactored Endpoint Example:

```
app.get("/allstudents", async (req, res) => {
```

```
    const students = await Student.find();
```

```
    res.json(students);
```

```
});
```

Frontend Update:

```
fetch(`${import.meta.env.VITE_API_URL}/allstudents`)
```

This demonstrated the ability to modify application routes while maintaining frontend-backend communication.

Data Synchronization Flow

User Opens Application



React Component Loads



useEffect Executes



Fetch API Request



Express Backend



MongoDB Database



JSON Response

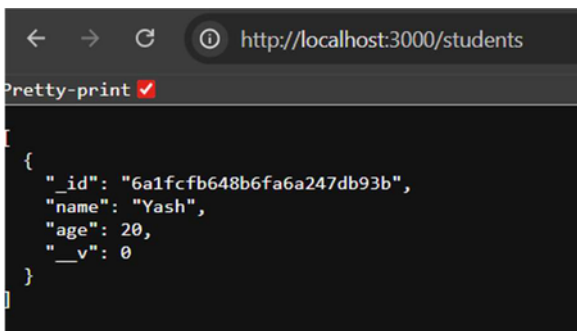


setStudents(data)

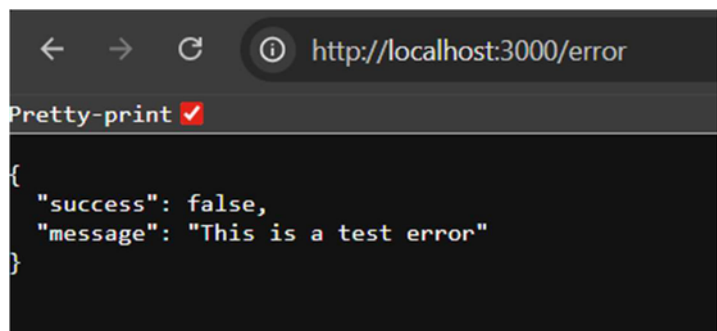


UI Updated

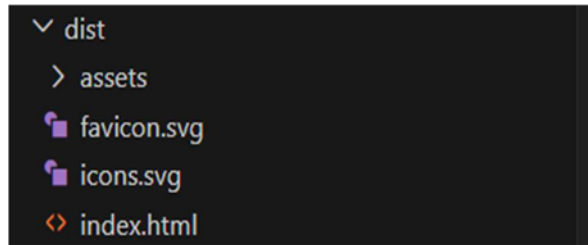
Task 6.1 Output Screenshots



```
http://localhost:3000/students  
Pretty-print ✓  
{  
  "_id": "6a1fcfb648b6fa6a247db93b",  
  "name": "Yash",  
  "age": 20,  
  "_v": 0  
}
```



```
http://localhost:3000/error  
Pretty-print ✓  
{  
  "success": false,  
  "message": "This is a test error"  
}
```



Task 6.1 Source Code

Frontend (App.jsx)

```
useEffect(() => {  
  
  fetch(`${import.meta.env.VITE_API_URL}/students`)  
    .then((response) => response.json())  
    .then((data) => {  
  
      setStudents(data);  
    })  
  }  
}, []);
```



```
});
```

```
},[]);
```

Backend (server.js)

```
app.get("/students", async (req, res) => {
```

```
    const students = await Student.find();
```

```
    res.json(students);
```

```
});
```

Conclusion

Day 6 focused on application validation, code review, and refactoring. The MERN application developed during previous sessions was successfully reviewed, including React Hooks, API communication, database integration, production build validation, and endpoint refactoring. The evaluation improved understanding of application architecture and strengthened the ability to explain and adapt existing code effectively.